

ADVANTAGEKIT

Data Flow

数据流

中英对照 · 8 篇文章

按照 AdvantageKit 文档顺序编排

Recording Inputs / 记录输入

Recording Outputs / 记录输出

Contents / 目录

01 Supported Types / 支持的数据类型	3
02 Built-In Logging / 内置日志记录	7
03 IO Interfaces / IO 接口 (记录输入)	12
04 Annotation Logging / 注解日志记录 (记录输入)	15
05 Dashboard Inputs / Dashboard 输入 (记录输入)	17
06 Recording Outputs / 记录输出 (记录输出)	19
07 Annotation Logging / 注解日志记录 (记录输出)	21
08 SysId Compatibility / SysId 兼容性	24

Supported Types / 支持的数据类型

Data is stored using string keys where slashes are used to denote subtables (similar to NetworkTables). Like NetworkTables, **all logged values are persistent (they will continue to appear on subsequent cycles until updated)**.

数据使用字符串键存储，其中斜杠用于表示子表（与 NetworkTables 类似）。与 NetworkTables 一样，**所有记录的值都会持续保留（在更新之前，它们会继续出现在后续周期中）**。

Simple / 简单类型

The following simple data types are currently supported:

目前支持以下简单数据类型：

- Single values: boolean, int, long, float, double, String
单个值: boolean, int, long, float, double, String
- Arrays: boolean[], int[], long[], float[], double[], String[], byte[]
数组: boolean[], int[], long[], float[], double[], String[], byte[]
- 2D Arrays: boolean[][] , int[][] , long[][] , float[][] , double[][] , String[][] , byte[][]
二维数组: boolean[][] , int[][] , long[][] , float[][] , double[][] , String[][] , byte[][]

Structured / 结构化类型

Many WPILib classes can be serialized to binary data using [structs](#) or [protobuffs](#). Supported classes include Translation2d, Pose3d, and SwerveModuleState with more coming soon. These classes can be logged as single values, arrays, or 2D arrays just like any simple type, and used as input or output fields.

许多 WPILib 类可以通过 [struct](#) 或 [protobuf](#) 序列化为二进制数据。支持的类包括 Translation2d、Pose3d 和 SwerveModuleState，后续还将支持更多类。这些类与任何简单类型一样，可以按单个值、数组或二维数组记录，并用作输入或输出字段。

DANGER / 危险

Protobuf logging can take an extended period (>100ms) the first time that a value with any given type is logged. Subsequent logging calls using an object of the same type will be significantly faster. **Protobuf values should always be logged for the first time when the robot is disabled.**

首次记录某个类型的值时，Protobuf 日志记录可能耗时较长 (>100ms)。之后使用相同类型对象的日志记录调用会快得多。**首次记录 Protobuf 值时，机器人应始终处于禁用状态。**

This issue is not applicable to struct logging, which is the default for all data types.

此问题不适用于 struct 日志记录；所有数据类型默认都使用 struct 日志记录。

Records / Record 类型

Custom [record](#) classes can be logged as structs, including support for single values, arrays, and 2D arrays as inputs or outputs. This enables efficient logging of custom complex data types, such as pose observations (check the [vision template](#) for examples).

自定义 [record](#) 类可以按 struct 记录，支持将单个值、数组和二维数组用作输入或输出。这使得位姿 (pose) 观测等自定义复杂数据类型能够被高效记录（示例请参阅[视觉模板](#)）。

Note that record fields must use only the following struct-compatible types. Array types are not supported for record fields. We recommend logging using multiple top-level record arrays as needed.

请注意，record 字段只能使用以下与 struct 兼容的类型。record 字段不支持数组类型。建议根据需要使用多个顶层 record 数组进行记录。

- Primitives: boolean, short, int, long, float, double
基本类型: boolean、short、int、long、float、double
- Enum values
枚举值
- Struct-compatible types (Pose2d, SwerveModuleState, etc.)
与 struct 兼容的类型 (Pose2d、SwerveModuleState 等)
- Record values (i.e. nested records)
Record 值 (即嵌套 record)

TIP / 提示

Logging multiple record types of the same name can cause conflicts. All record classes should be uniquely named.

记录多个同名 record 类型可能引发冲突。所有 record 类都应使用唯一的名称。

DANGER / 危险

Record logging can take an extended period (>100ms) the first time that a value with any given type is logged. Subsequent logging calls using an object of the same type will be significantly faster. **Record values should always be logged for the first time when the robot is disabled.**

首次记录某个类型的值时，record 日志记录可能耗时较长 (>100ms)。之后使用相同类型对象的日志记录调用会快得多。**首次记录 record 值时，机器人应始终处于禁用状态。**

Units / 单位

AdvantageKit includes extensive support for unit-safe logging, including compatibility with AdvantageScope's [unit visualization](#) and the WPILib [units library](#). See the sections below for more information:

AdvantageKit 广泛支持具有单位安全性的日志记录，包括兼容 AdvantageScope 的[单位可视化](#)和 WPILib 的[单位库](#)。更多信息请参阅以下章节：

- Input logging ([link](#))
输入日志记录 ([链接](#))
- Output logging ([link](#))
输出日志记录 ([链接](#))
- Output annotation logging ([link](#))
使用注解记录输出 ([链接](#))

INFO / 说明

In WPILib, measure implementations (like Angle, Distance, etc.) are defined as Java records. These record-based measures can be passed to the generic `put()`, `get()`, or `recordOutput()` methods. If a variable is declared using the interface type directly (e.g. `Measure<AngleUnit>`), you must use the explicit measure-specific methods: `putMeasure()`, `getMeasure()`, and `recordOutputMeasure()`.

在 WPILib 中，测量值的实现（如 Angle、Distance 等）被定义为 Java record。这些基于 record 的测量值可以传入通用的 `put()`、`get()` 或 `recordOutput()` 方法。如果变量直接使用接口类型声明（例如 `Measure<AngleUnit>`），则必须使用专门针对测量值的显式方法：`putMeasure()`、`getMeasure()` 和 `recordOutputMeasure()`。

Colors / 颜色

WPILib includes a [color library](#) that can be used to simplify color operations. These values will be stored in the log as a string formatted in [Hex Triplet](#) color notation.

WPILib 包含一个可用于简化颜色操作的颜色库。这些值会以采用十六进制三元组颜色表示法的字符串存储在日志中。

Enums / 枚举

`Enum` values can be logged and replayed by AdvantageKit. These values will be stored in the log as string values (using the `name()` method).

AdvantageKit 可以记录和回放枚举值。这些值会以字符串值存储在日志中（使用 `name()` 方法）。

Suppliers (Output Only) / Supplier (仅输出)

Primitive suppliers (`BooleanSupplier`, `IntSupplier`, `LongSupplier`, and `DoubleSupplier`) can be used in place of their single values for output logging, including annotation logging with `@AutoLogOutput`. One application of this feature is logging `Trigger` values, which extend from `BooleanSupplier`.

记录输出时，可以使用基本类型 supplier (`BooleanSupplier`、`IntSupplier`、`LongSupplier` 和 `DoubleSupplier`) 来代替相应的单个值，包括使用 `@AutoLogOutput` 的注解日志记录。这一功能的一种应用是记录继承自 `BooleanSupplier` 的 `Trigger` 的值。

Mechanisms (Output Only) / 机构 (仅输出)

AdvantageKit can log 2D mechanism objects as outputs, which can be viewed using `AdvantageScope`. If not using `@AutoLogOutput`, note that the logging call only records the current state of the `Mechanism2d` and so it must be called periodically.

AdvantageKit 可以将二维机构对象记录为输出，并使用 `AdvantageScope` 查看。如果不使用 `@AutoLogOutput`，请注意，日志记录调用只会记录 `Mechanism2d` 的当前状态，因此必须周期性调用。

TIP / 提示

The `generate3dMechanism()` method can be used to convert a `Mechanism2d` to an array of `Pose3d` objects compatible with [articulated components](#) in `AdvantageScope`.

`generate3dMechanism()` 方法可以将 `Mechanism2d` 转换为 `Pose3d` 对象数组，以兼容 `AdvantageScope` 中的关节连接组件。

WARNING / 警告

Mechanism objects must use the `LoggedMechanism2d` class to be compatible with AdvantageKit. This class is otherwise equivalent to the standard `Mechanism2d` class. Equivalent `LoggedMechanismRoot2d`, `LoggedMechanismObject2d`, and `LoggedMechanismLigament2d` classes are also provided.

机构对象必须使用 `LoggedMechanism2d` 类才能兼容 AdvantageKit。除此之外，该类与标准的 `Mechanism2d` 类等效。还提供了等效的 `LoggedMechanismRoot2d`、`LoggedMechanismObject2d` 和 `LoggedMechanismLigament2d` 类。

```
public class Example {
    @AutoLogOutput // Auto logged as "Example/Mechanism"
    private LoggedMechanism2d mechanism = new LoggedMechanism2d(3, 3);

    public void periodic() {
        // Alternative approach if not using @AutoLogOutput
        // (Must be called periodically)
        Logger.recordOutput("Example/Mechanism", mechanism);

        // Log Pose3d objects for articulated components in AdvantageScope
        Logger.recordOutput("Example/MechanismPoses", mechanism.generate3dMechanisms());
    }
}
```

译注：原文代码使用了 `generate3dMechanisms()`，但上文及本地 AdvantageKit 源码中的方法名为 `generate3dMechanism()`；代码按原文保留。

Built-In Logging / 内置日志记录

AdvantageKit automatically logs many important fields as inputs or outputs. No configuration is required to use these features.

AdvantageKit 会自动将许多重要字段记录为输入或输出。使用这些功能无需任何配置。

INFO / 说明

Built-in input values are available during replay with guaranteed accuracy and can be freely accessed in user code without manual logging. **All other inputs must be logged using an IO interface.**

内置输入值在回放期间可用且保证准确，可以在用户代码中直接访问，无需手动记录。**所有其他输入都必须使用 IO 接口记录。**

Inputs / 输入

Timestamp / 时间戳

AdvantageKit logs and replays the value of `RobotController.getTime()`, `Timer.getTimestamp()`, etc. See [this page](#) for more details on timestamps in AdvantageKit.

AdvantageKit 会记录和回放 `RobotController.getTime()`、`Timer.getTimestamp()` 等的值。有关 AdvantageKit 中时间戳的更多细节，请参阅[此页面](#)。

WARNING / 警告

The methods `RobotController.getMonotonicTime()` and `Timer.getMonotonicTimestamp()` are used for accessing the real (non-deterministic) timestamp, and should only be used within IO implementations or for performance profiling.

`RobotController.getMonotonicTime()` 和 `Timer.getMonotonicTimestamp()` 方法用于访问真实的（非确定性的）时间戳，只应在 IO 实现内部或性能分析中使用。

Driver Station / 驾驶站

All values that can be accessed via the `DriverStation` or `WPILib` HID classes (`Joystick`, `XboxController`, etc) are automatically logged and replayed. These fields are available under the `DriverStation` table.

所有可以通过 `DriverStation` 或 `WPILib` HID 类（`Joystick`、`XboxController` 等）访问的值都会自动记录和回放。这些字段位于 `DriverStation` 表下。

DANGER / 危险

The `waitForDsConnection` method is not compatible with AdvantageKit.

`waitForDsConnection` 方法与 AdvantageKit 不兼容。

Dashboard Inputs / Dashboard 输入

Dashboard inputs accessed via **AdvantageKit dashboard classes** are automatically logged and replayed. See [this page](#) for details. These fields are available under the NetworkInputs table.

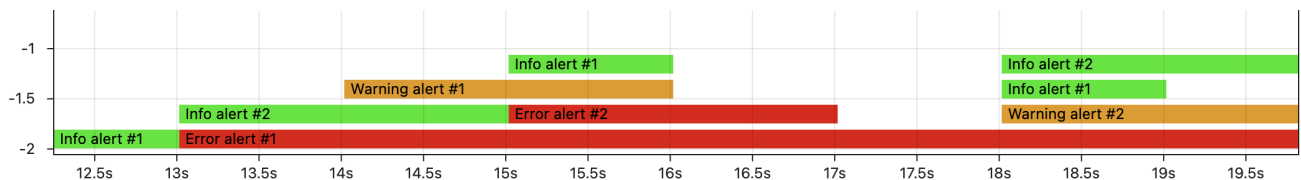
通过 **AdvantageKit dashboard** 类访问的 dashboard 输入会自动记录和回放。详情请参阅[此页面](#)。这些字段位于 NetworkInputs 表下。

Outputs / 输出

Alerts / 警报

The state of any alerts created through WPILib's [persistent alerts](#) API will be automatically logged as outputs. These alerts can be visualized using AdvantageScope's [Line Graph](#) tab. These fields are available under the RealOutputs OR ReplayOutputs table.

通过 WPILib 的[持久警报](#) API 创建的所有警报，其状态都会自动记录为输出。这些警报可以使用 AdvantageScope 的[折线图](#)选项卡进行可视化。这些字段位于 RealOutputs 或 ReplayOutputs 表下。



Alerts screenshot / 警报截图

Console / 控制台

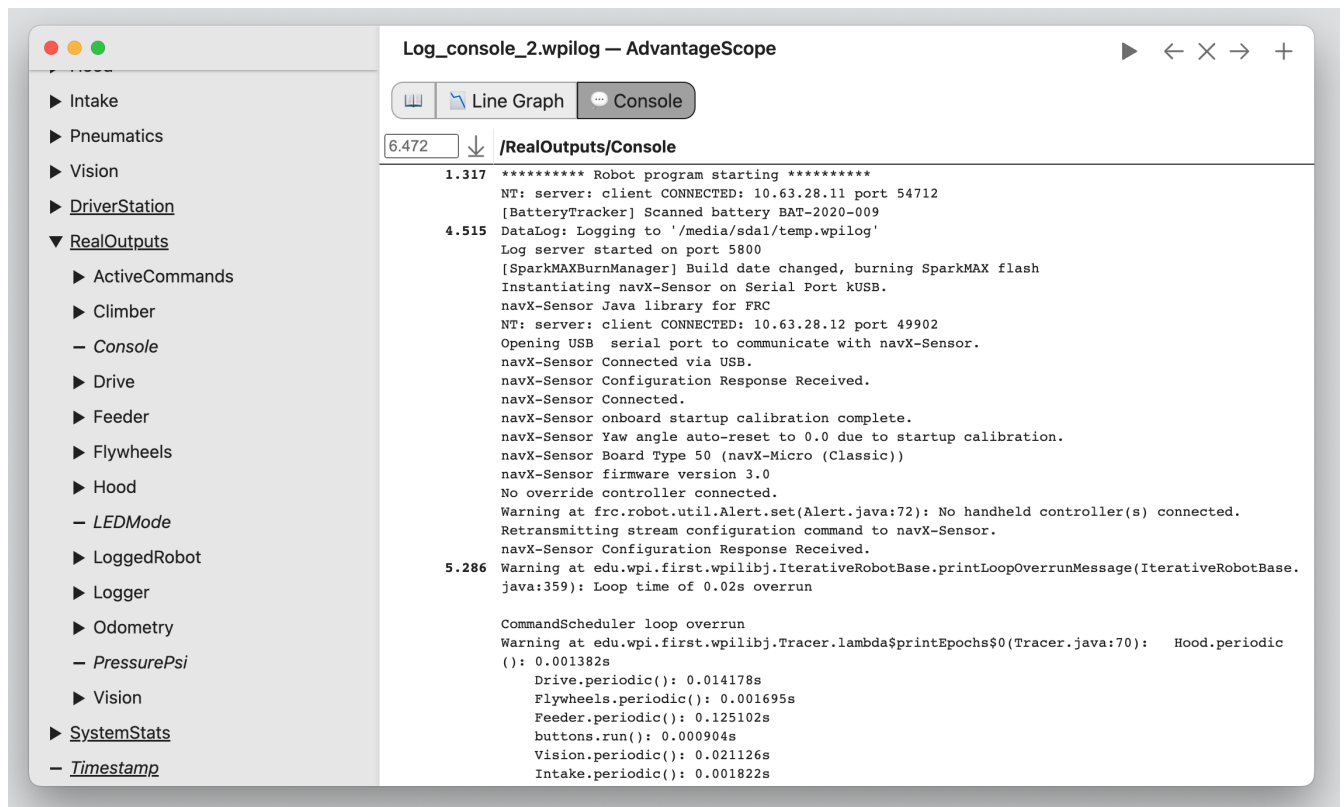
Console output is automatically logged by AdvantageKit to the Console field, and can be viewed using AdvantageScope's [Console](#) tab. This field is available under the RealOutputs OR ReplayOutputs table.

AdvantageKit 会自动将控制台输出记录到 Console 字段，可以使用 AdvantageScope 的[控制台](#)选项卡查看。该字段位于 RealOutputs 或 ReplayOutputs 表下。

INFO / 说明

Output from native code is not included when running in simulation.

在仿真中运行时，不包含来自原生代码的输出。

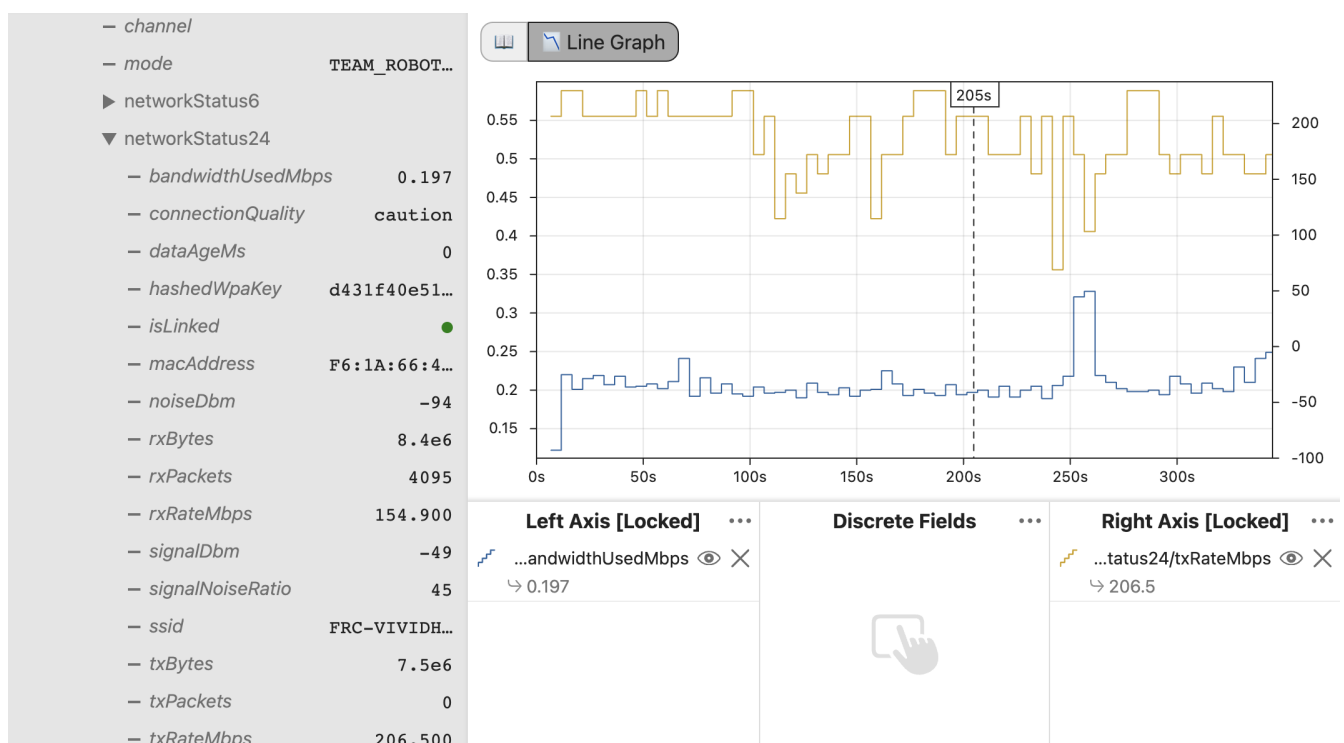


Console screenshot / 控制台截图

Radio Status / 无线电状态

Status data from the VH-109 radio is automatically logged every ~5 seconds. This includes useful information about the connection status, bandwidth usage, etc. These fields are available under the RadioStatus table.

VH-109 无线电的状态数据大约每 5 秒自动记录一次。其中包括连接状态、带宽使用情况等有用信息。这些字段位于 RadioStatus 表下。



Radio data / 无线电数据

Power Distribution Data / 配电数据

The current on each channel, along with other useful stats, are automatically logged by AdvantageKit as outputs under the PowerDistribution table. This feature works by default when using the CTRE PDP or REV PDH configured to the default CAN ID (ID 0 for the PDP and ID 1 for the PDH).

AdvantageKit 会自动将各通道的电流及其他有用的统计信息记录为输出，存放在 PowerDistribution 表下。当使用配置为默认 CAN ID 的 CTRE PDP 或 REV PDH 时，此功能默认即可使用（PDP 的 ID 为 0，PDH 的 ID 为 1）。

WARNING / 警告

This feature is only supported on Power Distribution devices that support current monitoring (the CTRE PDP and REV PDH). The CTRE PDP 2.0 and AndyMark Power Distribution Board are **not supported**.

此功能仅支持具备电流监测功能的配电设备（CTRE PDP 和 REV PDH）。CTRE PDP 2.0 和 AndyMark Power Distribution Board **不受支持**。

The LoggedPowerDistribution class can be used to manually configure the power distribution type and CAN ID when not using the default configuration. This can be accomplished by adding the line below to the Robot constructor before Logger.start():

不使用默认配置时，可以使用 LoggedPowerDistribution 类手动配置配电设备类型和 CAN ID。为此，可在 Robot 构造函数中的 Logger.start() 之前添加以下代码行：

```
LoggedPowerDistribution.getInstance(50, ModuleType.kRev); // Example: PDH on CAN ID 50
```

System Stats / 系统统计信息

Important status information from the roboRIO is automatically recorded, such as the battery voltage, rail status, CAN status, system time, and NT client connections. These fields are available under the SystemStats table.

来自 roboRIO 的重要状态信息会自动记录，例如电池电压、电源轨状态、CAN 状态、系统时间和 NT 客户端连接。这些字段位于 SystemStats 表下。

Performance Data / 性能数据

Several important fields are automatically recorded to measure the performance of the robot code:

以下几个重要字段会被自动记录，用于衡量机器人代码的性能：

- LoggedRobot/FullCycleMS:** The execution time of all periodic code, should be less than the loop period (20ms by default).
 LoggedRobot/FullCycleMS: 所有周期性代码的执行时间，应小于循环周期（默认 20ms）。
- LoggedRobot/UserCodeMS:** The execution time of all user periodic code.
 LoggedRobot/UserCodeMS: 所有用户周期性代码的执行时间。
- LoggedRobot/LogPeriodicMS:** The execution time of all AdvantageKit periodic code.
 LoggedRobot/LogPeriodicMS: 所有 AdvantageKit 周期性代码的执行时间。
- LoggedRobot/GCTimeMS:** The total execution time of the Java garbage collector within the last loop cycle, may or may not overlap with other code execution.
 LoggedRobot/GCTimeMS: Java 垃圾回收器在上一个循环周期内的总执行时间，可能与其他代码执行重叠，也可能不重叠。

- `LoggedRobot/GCCount`: The total number of collections performed by the Java garbage collector within the last loop cycle.
`LoggedRobot/GCCount`: Java 垃圾回收器在上一个循环周期内执行的回收总次数。
- `Logger/QueuedCycle`: The number of cycles of data in queue to be written to data receivers.
`Logger/QueuedCycle`: 队列中等待写入数据接收器的数据周期数量。
- `Logger/...MS`: The execution time of each step of the AdvantageKit periodic code.
`Logger/...MS`: AdvantageKit 周期性代码各步骤的执行时间。

译注：原文中的 `LoggedRobot/UserCodeMs` 和 `Logger/QueuedCycle` 与本地 AdvantageKit 源码实际记录的键名不一致；实际键名分别为 `LoggedRobot/UserCodeMS` 和 `Logger/QueuedCycles`。上方条目按原文保留。

IO Interfaces / IO 接口

By necessity, any interaction with external hardware must be isolated such that all input data is logged and can be replayed in the simulator where that hardware is not present. Most hardware interaction occurs in subsystem classes (read [this section](#) for information on using NetworkTables as an input). Traditionally, a subsystem has three main components:

必须隔离与外部硬件的所有交互，以便记录全部输入数据，并在不存在这些硬件的仿真器中回放。大多数硬件交互发生在 subsystem 类中（有关将 NetworkTables 用作输入的信息，请阅读[此章节](#)）。传统的 subsystem 有三个主要组成部分：

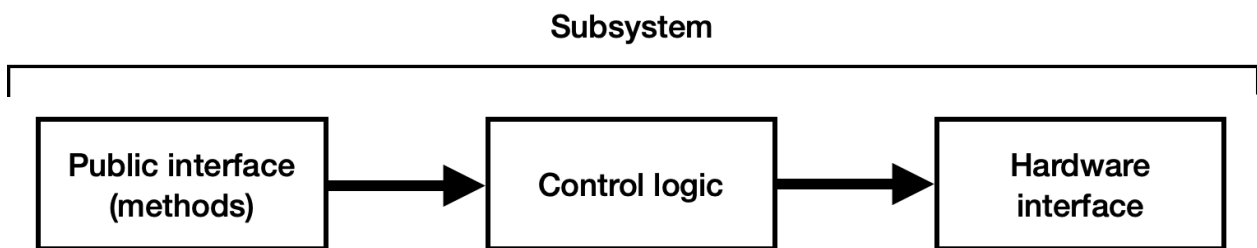


Diagram of traditional subsystem / 传统 subsystem 结构图

- The **public interface** consists of methods used by the rest of the robot code to control the subsystem.
公共接口由机器人其余代码用来控制 subsystem 的方法组成。
- The **control logic** is the internal code used to follow those commands or analyze sensor data.
控制逻辑是用于执行这些指令或分析传感器数据的内部代码。
- The **hardware interface** is the code used to read sensors and directly control hardware like motors or pneumatics.
硬件接口是用于读取传感器并直接控制电机或气动装置等硬件的代码。

Data logging of inputs should occur between the control logic and hardware interface - this ensures that any control logic can be replayed in the simulator. We suggest restructuring the subsystem such that hardware interfacing occurs in a separate object (we call this the "IO" layer). The IO layer includes an interface defining all methods used for interacting with the hardware along with one or more implementations that make use of vendor libraries to carry out commands and read data.

输入数据的日志记录应发生在控制逻辑与硬件接口之间，这样可以确保任何控制逻辑都能在仿真器中回放。建议调整 subsystem 的结构，将硬件交互放在一个独立对象中（我们称之为“IO”层）。IO 层包含一个接口，定义所有用于与硬件交互的方法，以及一个或多个使用 vendor library 执行指令和读取数据的实现。

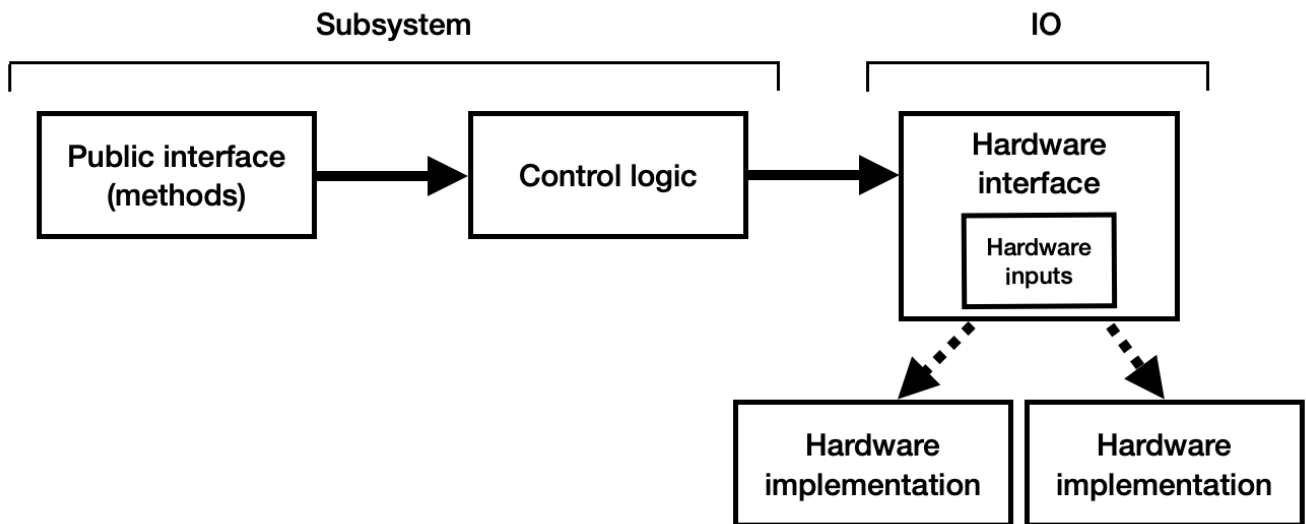


Diagram of restructured subsystem / 调整后的 subsystem 结构图

TIP / 提示

Refer to the [AdvantageKit templates](#) for some reference IO interfaces and implementations.

一些可供参考的 IO 接口及实现，请参阅 [AdvantageKit 模板](#)。

Outputs (setting voltage, setpoint, PID constants, etc.) make use of simple methods for each command. Input data is more controlled such that it can be logged and replayed. Each IO interface defines a class with public attributes for all input data, along with methods for saving and replaying that data from a log (toLog and fromLog). We recommend using the `@AutoLog` annotation to generate these methods automatically.

输出（设置电压、setpoint、PID 常量等）通过对应于各项指令的简单方法实现。输入数据受到更严格的管理，以便记录和回放。每个 IO 接口都定义一个类，使用 public 字段保存全部输入数据，并提供将这些数据保存到日志以及从日志回放的方法（toLog 和 fromLog）。建议使用 `@AutoLog` 注解自动生成这些方法。

The IO layer includes a single method (updateInputs) for updating all of the input data. The subsystem class contains an instance of both the current IO implementation and the "inputs" object. Once per cycle, it updates the input data and sends it to the logging framework:

IO 层包含一个用于更新全部输入数据的方法（updateInputs）。subsystem 类同时持有当前 IO 实现的实例和“inputs”对象的实例。每个周期，它更新一次输入数据并将其发送给日志框架：

```
io.updateInputs(inputs); // Update input data from the IO layer
Logger.processInputs("ExampleSubsystem", inputs); // Send input data to the logging framework (or update from the log during replay)
```

The rest of the subsystem then reads data from this inputs object rather than directly from the IO layer. This structure ensures that:

随后，subsystem 的其余部分从该 inputs 对象读取数据，而不是直接从 IO 层读取。这种结构可以确保：

- The logging framework has access to all of the data being logged and can insert data from the log during replay.

日志框架能够访问全部被记录的数据，并在回放期间插入来自日志的数据。

- Throughout each cycle, all code making use of the input data reads the same values - the cache is never updated during a cycle. This means that the data replayed from the log appears identical to the data read on the real robot.

在每个周期内，所有使用输入数据的代码都读取相同的值；缓存绝不会 在一个周期的执行过程中 更新。这意味着，从日志回放的数据与在真实机器人上读取的数据表现完全相同。

All of the IO methods include a default implementation which is used during simulation. We suggest setting up each subsystem accept the IO object as a constructor argument, so that the central robot class (like RobotContainer) can decide whether or not to use real hardware:

所有 IO 方法都包含一个用于仿真期间的默认实现。建议让每个 subsystem 通过构造函数参数接收 IO 对象，以便机器人中央类（如 RobotContainer）决定是否使用真实硬件：

```
public RobotContainer() {
    if (isReal()) {
        // Instantiate IO implementations to talk to real hardware
        driveTrain = new DriveTrain(new DriveTrainIOReal());
        elevator = new Elevator(new ElevatorIOReal());
        intake = new Intake(new IntakeIOReal());
    } else {
        // Use anonymous classes to create "dummy" IO implementations
        driveTrain = new DriveTrain(new DriveTrainIO() {});
        elevator = new Elevator(new ElevatorIO() {});
        intake = new Intake(new IntakeIO() {});
    }
}
```

TIP / 提示

We suggest the use of an IO layer to minimize the chance of interacting with hardware that doesn't exist. However, any structure will work where all input data flows through an inputs object implementing LoggableInputs and the two methods fromLog and toLog. Feel free to make use of whatever structure best fits your own requirements.

建议使用 IO 层，以尽量减少与不存在的硬件交互的可能性。不过，只要所有输入数据都流经一个实现了 LoggableInputs 以及 fromLog 和 toLog 两个方法的 inputs 对象，任何结构都可以使用。可以自由选择最符合自身需求的结构。

Annotation Logging / 注解日志记录

By adding the `@AutoLog` annotation to your inputs class, AdvantageKit will automatically generate implementations of `toLog` and `fromLog` for your inputs. All [data types](#) are supported with the exception of mechanism states. Loggable inputs can also be nested and used as fields.

在 inputs 类上添加 `@AutoLog` 注解后, AdvantageKit 会自动为输入生成 `toLog` 和 `fromLog` 的实现。除了机构状态之外, 所有数据类型都受支持。可记录的输入也可以嵌套, 并用作字段。

For example:

例如:

```
@AutoLog
public class MyInputs {
    public double myNumber = 0.0;
    public Pose2d myPose = new Pose2d();
    public MyEnum myEnum = MyEnum.VALUE;
}
```

This will generate the following class:

这将生成以下类:

```
class MyInputsAutoLogged extends MyInputs implements LoggableInputs {
    public void toLog(LogTable table) {
        table.put("MyNumber", myField);
        table.put("MyPose", myPose);
        table.put("MyEnum", myEnum);
    }

    public void fromLog(LogTable table) {
        myNumber = table.get("MyNumber", myNumber);
        myPose = table.get("MyPose", myPose);
        myEnum = table.get("MyEnum", myEnum);
    }
}
```

译注: 原文示例中的 `myField` 与前面声明的字段名不一致; 根据 `MyInputs` 的定义, 此处应为 `myNumber`。代码按原文保留。

Note that you should use the `<className>AutoLogged` class, rather than your annotated class. The [AdvantageKit template projects](#) are a useful reference for how to use `@AutoLog` in a full project.

请注意, 应使用 `<className>AutoLogged` 类, 而不是添加了注解的原始类。AdvantageKit 模板项目可用于参考如何在完整项目中使用 `@AutoLog`。

Units / 单位

To ensure that AdvantageScope will correctly [visualize unit data](#), units can be specified by modifying the field name or using a Measure object.

为确保 AdvantageScope 能正确可视化带单位的数据, 可以通过修改字段名或使用 Measure 对象来指定单位。

INFO / 说明

Unlike when logged as [outputs](#), Measure values in an inputs class are always logged using the **base unit** and not the user-specified unit (e.g. distances will always be logged in meters). This ensures that unit differences between IO implementations are not reflected in the logged data.

与记录为[输出](#)时不同，inputs 类中的 Measure 值始终使用**基本单位**记录，而不是用户指定的单位（例如，距离始终以米记录）。这可以确保 IO 实现之间的单位差异不会体现到记录的数据中。

```
@AutoLog
public class MyInputs {
    public double myDistanceMeters = 0.0;
    public Distance myDistance = Meters.of(0.0);
}
```

Dashboard Inputs / Dashboard 输入

Like the robot's hardware, **data retrieved from NetworkTables must be isolated and treated as input data.** For example, the following call will NOT function correctly in replay:

与机器人硬件一样，**从 NetworkTables 获取的数据必须被隔离，并作为输入数据处理。** 例如，以下调用在回放中无法正常工作：

```
var flywheelSetpoint = SmartDashboard.getNumber("FlywheelSpeed", 0.0);
```

AdvantageKit provides several solutions to deal with this issue:

AdvantageKit 提供了几种解决此问题的方案：

- For subsystems that use NT input data (reading from coprocessors), we recommend treating the NetworkTables interaction as a hardware interface using an IO layer. See the [vision template project](#) as an example.
对于使用 NT 输入数据（从协处理器读取）的 subsystem，建议将 NetworkTables 交互视为硬件接口，使用 IO 层进行处理。示例请参阅[视觉模板项目](#)。
- When reading dashboard inputs from NT (auto selector, tuning values, etc) AdvantageKit includes the following classes that correctly handle periodic logging and replay:
通过 NT 读取 dashboard 输入（自动程序选择器、调参值等）时，AdvantageKit 包含以下能够正确处理周期性日志记录和回放的类：
 - [LoggedNetworkChooser](#) - Replaces Selectable with equivalent functionality. See the example below.
[LoggedNetworkChooser](#): 替代 Selectable，提供等效功能。参见下方示例。
 - [LoggedNetworkNumber](#) - Simple number field
[LoggedNetworkNumber](#): 简单的数值字段
 - [LoggedNetworkString](#) - Simple string field
[LoggedNetworkString](#): 简单的字符串字段
 - [LoggedNetworkBoolean](#) - Simple boolean field
[LoggedNetworkBoolean](#): 简单的布尔字段

Example use of LoggedNetworkChooser for auto routines in a command-based project:

在 Command-Based 项目中使用 LoggedNetworkChooser 选择自动程序的示例：

```
private final LoggedNetworkChooser<Command> autoChooser = new LoggedNetworkChooser<>("Auto Routine");

public RobotContainer() {
    // ...
    autoChooser.addDefault("Do Nothing", new InstantCommand());
    autoChooser.add("My First Auto", new MyFirstAuto());
    autoChooser.add("My Second Auto", new MySecondAuto());
    autoChooser.add("My Third Auto", new MyThirdAuto());
}

public Command getAutonomousCommand() {
    return autoChooser.get();
}
```

TIP / 提示

AdvantageScope supports tuning via NetworkTables when running in the AdvantageKit NetworkTables mode. Tunable values must be published to the "/Tuning" table using `LoggedNetworkNumber`, `LoggedNetworkString`, or `LoggedNetworkBoolean`. Check the [AdvantageScope docs](#) for details.

在 AdvantageKit NetworkTables 模式下运行时，AdvantageScope 支持通过 NetworkTables 调参。可调参数必须使用 `LoggedNetworkNumber`、`LoggedNetworkString` 或 `LoggedNetworkBoolean` 发布到 "/Tuning" 表。详情请查看 [AdvantageScope 文档](#)。

A `LoggedNetworkChooser` can also be constructed using an existing `Selectable`, which allows for compatibility with PathPlanner's AutoBuilder API:

也可以使用现有的 `Selectable` 构造 `LoggedNetworkChooser`，从而兼容 PathPlanner 的 AutoBuilder API:

```
private final LoggedNetworkChooser<Command> autoChooser;

public RobotContainer() {
    // ...

    // buildAutoChooser() returns a Selectable
    autoChooser = new LoggedNetworkChooser<>("Auto Routine", AutoBuilder.buildAutoChooser());
}
```

Recording Outputs / 记录输出

Output data consists of any calculated values which could be recreated in the simulator, including...

输出数据包括任何可以在仿真器中重新生成的计算值，例如.....

- Odometry pose
里程计 (odometry) 位姿 (pose)
- Motor voltages
电机电压
- Pneumatics commands
气动控制指令
- Status data for drivers
提供给驾驶员的状态数据
- Internal object state
对象内部状态

The logging framework supports recording this output data on the real robot and during replay. Essential data like the odometry pose are recorded on the real robot for convenience; even if it can be recreated in a simulator, that's often not a viable option in the rush to fix a problem between matches. During replay, recording extra output data is the primary method of debugging the code - logging calls can be added anywhere as they don't interfere with the replayed control logic. Any loggable data type ([see here](#)) can be saved as an output like so:

日志框架支持在真实机器人上以及回放期间记录这些输出数据。为方便使用，会在真实机器人上记录 odometry pose 等关键数据；即使这些数据能够在仿真器中重新生成，在比赛间隙急于修复问题时，这通常也不可行。在回放期间，记录额外的输出数据是调试代码的主要方法；日志记录调用不会干扰被回放的控制逻辑，因此可以添加到任意位置。任何可记录的数据类型 ([参见此处](#)) 都可以像下面这样保存为输出：

```
Logger.recordOutput("Flywheel/Setpoint", setpointSpeed);
Logger.recordOutput("Drive/Pose", odometryPose);
Logger.recordOutput("FeederState", FeederState.RUNNING);
```

INFO / 说明

This data is automatically saved to the RealOutputs or ReplayOutputs table, and it can be divided further into subtables using slashes (as seen above).

这些数据会自动保存至 RealOutputs 或 ReplayOutputs 表中，还可以使用斜杠进一步划分为子表（如上所示）。

Structured Types / 结构化类型

Logging geometry objects like Pose2d, Trajectory, etc. is common in robot code. Many WPILib classes can be serialized to binary data using [structs](#) or [protobufs](#). These objects can be logged as single values or arrays:

在机器人代码中，记录 Pose2d、Trajectory 等几何对象很常见。许多 WPILib 类可以通过 [struct](#) 或 [protobuf](#) 序列化为二进制数据。这些对象可以按单个值或数组记录：

```
// Pose2d
Pose2d poseA, poseB, poseC;
Logger.recordOutput("MyPose2d", poseA);
Logger.recordOutput("MyPose2dArray", poseA, poseB);
Logger.recordOutput("MyPose2dArray", new Pose2d[] { poseA, poseB });

// Pose3d
Pose3d poseA, poseB, poseC;
Logger.recordOutput("MyPose3d", poseA);
Logger.recordOutput("MyPose3dArray", poseA, poseB);
Logger.recordOutput("MyPose3dArray", new Pose3d[] { poseA, poseB });

// Trajectory
Trajectory trajectory;
Logger.recordOutput("MyTrajectory", trajectory);

// SwerveModuleState
SwerveModuleState stateA, stateB, stateC, stateD;
Logger.recordOutput("MySwerveModuleStates", stateA, stateB, stateC, stateD);
Logger.recordOutput("MySwerveModuleStates", new SwerveModuleState[] { stateA, stateB, stateC, stateD });
```

Units / 单位

Double or float fields can be logged with unit metadata in multiple ways. This ensures that AdvantageScope will correctly **visualize unit data**.

double 或 float 字段可以通过多种方式附带单位元数据进行记录。这可以确保 AdvantageScope 正确可视化带单位的数据。

```
// The unit can be specified as a string (stored as metadata)
Logger.recordOutput("MyDistance", 3.14, "meters");

// A unit object from the WPILib units library can be used in place of a string
Logger.recordOutput("MyDistance", 3.14, Meters);

// Measure values will also be saved with unit metadata
// (The raw value will use the user-specified unit, not the base unit)
Logger.recordOutputMeasure("MyDistance", Meters.of(3.14));

// This works too, but requires adding the unit to the field name
// (See the AdvantageScope docs linked above for details)
Logger.recordOutput("MyDistanceMeters", 3.14);
```

Annotation Logging / 注解日志记录

The `@AutoLogOutput` annotation can also be used to automatically log the value of a field or getter method as an output periodically (including private fields and methods). The key will be selected automatically, or it can be overridden using the key parameter. All data types are supported, including arrays and structured data types.

`@AutoLogOutput` 注解也可以用于周期性地将字段或 getter 方法的值自动记录为输出（包括 private 字段和方法）。键会自动选择，也可以通过 key 参数覆盖。所有数据类型都受支持，包括数组和结构化数据类型。

```
public class Example {
    @AutoLogOutput // Logged as "Example/MyPose"
    private Pose2d myPose = new Pose2d();

    @AutoLogOutput(key = "Custom/Speeds")
    public double[] getSpeeds() {...}
}
```

Parameters / 参数

Key / 键

The key parameter can reference other fields within the same class using the syntax shown below. This is useful to disambiguate classes with multiple instances, such as swerve modules. The value of the referenced field will not be updated after the first loop cycle. Any data type convertible to a string is supported, including numbers, booleans, and strings.

key 参数可以使用下方所示语法引用同一类中的其他字段。这有助于区分具有多个实例的类，例如 swerve 模块。第一个循环周期之后，用于键的被引用字段值不再更新。支持任何可转换为字符串的数据类型，包括数值、布尔值和字符串。

```
public class SwerveModule {
    private final int index; // 0, 1, 2, or 3
    private final String descriptor; // "FL", "FR", "BL", "BR"

    @AutoLogOutput(key = "Module{index}/Speed") // e.g. "Module0/Speed"
    public double getSpeed() {...}

    @AutoLogOutput(key = "Odometry/ModulePose{descriptor}") // e.g. "Odometry/ModulePoseFL"
    public Pose2d getPose() {...}
}
```

Unit / 单位

For double or float fields, the unit parameter can be used to provide a unit name that will be stored as metadata. This ensures that AdvantageScope will correctly [visualize unit data](#). Note that Measure values can also be logged using `@AutoLogOutput` with full support for unit metadata.

对于 double 或 float 字段，可以使用 unit 参数提供一个单位名称，将其存储为元数据。这可以确保 AdvantageScope 正确可视化带单位的数据。请注意，Measure 值也可以使用 `@AutoLogOutput` 记录，并完整支持单位元数据。

```
public class Example {
    // AdvantageScope will plot this value using meters
    @AutoLogOutput(unit = "meters")
    private double myDistance = 0.0;

    // This line is equivalent, if using the WPILib units library
    // (The raw value will use the user-specified unit, not the base unit)
    @AutoLogOutput
    private Distance myDistanceMeasure = Meters.of(0.0);

    // This works too, but requires adding the unit to the field name
    // (See the AdvantageScope docs linked above for details)
    @AutoLogOutput
    private double myDistanceMeters = 0.0;
}
```

Force Serializable / 强制序列化

The `forceSerializable` parameter can be used to force the use of struct or Protobuf serialization for enum types, as shown in the example below. The class should inherit from `StructSerializable`, `ProtobufSerializable`, or `WPISerializable`.

`forceSerializable` 参数可以强制对枚举类型使用 struct 或 Protobuf 序列化，如下方示例所示。该类应实现 `StructSerializable`、`ProtobufSerializable` 或 `WPISerializable` 接口。

```
enum CustomEnum implements StructSerializable {
    A(...),
    B(...);

    public final double value;
    public static final Struct<ElevatorSetpoints> struct = ...;

    Setpoint(double value) {
        this.value = value;
    }
}

@AutoLogOutput
CustomEnum setpoint = CustomEnum.A; // Logs as an enum

@AutoLogOutput(forceSerializable = true)
CustomEnum setpoint = CustomEnum.A; // Logs as a struct
```

译注：原文示例中的构造函数名 `Setpoint` 和 `Struct<ElevatorSetpoints>` 的类型参数与 `CustomEnum` 不一致；在该示例中应分别为 `CustomEnum` 和 `Struct<CustomEnum>`。代码按原文保留。

Global Configuration / 全局配置

By default, the parent class where `@AutoLogOutput` is used must be within the same package as `Robot` (or a subpackage). The following method can be called in the constructor of `Robot` to allow additional packages, such as a "lib" package outside of normal robot code:

默认情况下，使用 `@AutoLogOutput` 的所属类必须与 `Robot` 位于同一个 package（或其子 package）中。可以在 `Robot` 的构造函数中调用以下方法，以允许额外的 package，例如常规机器人代码之外的 "lib" package：

```
AutoLogOutputManager.addPackage("frc.lib");
```

The `addObject` method can also be used to manually scan an object for loggable fields. This method should only be called during initialization:

也可以使用 `addObject` 方法手动扫描对象中的可记录字段。此方法只应在初始化期间调用：

```
AutoLogOutputManager.addObject(this);
```

WARNING / 警告

The parent class where `@AutoLogOutput` is used must also be instantiated within the first loop cycle and be accessible by a recursive search of the fields of `Robot`. This feature is primarily intended to log outputs from subsystems and other similar classes. For classes that do not fit the criteria above, call `Logger.recordOutput` periodically to record outputs.

使用 `@AutoLogOutput` 的所属类还必须在第一个循环周期内实例化，并且能够通过递归搜索 `Robot` 的字段访问到。这一功能主要用于记录 subsystem 及其他类似类的输出。对于不符合上述条件的类，应周期性调用 `Logger.recordOutput` 来记录输出。

SysId Compatibility / SysId 兼容性

WPILib provides tools to perform [system identification](#) on robot mechanisms, enabling [feedforward](#) and [feedback](#) controller gains to be calculated based on real-world data. Starting in 2024, identification routines are defined in user code as described [here](#). Data is recorded to a WPILOG file for analysis in the SysId application.

WPILib 提供工具，可对机器人机构执行 [SysId](#)，从而根据实际数据计算前馈（[feedforward](#)）和反馈控制器的增益。从 2024 年开始，SysId 测试程序在用户代码中定义，方式如[此处](#)所述。数据记录到 WPILOG 文件中，以便在 SysId 应用程序中分析。

Since AdvantageKit already requires subsystems to log relevant sensor data, setting up identification routines in user code is simplified considerably. This document outlines how the process of collecting SysId data differs when using AdvantageKit for data logging. **Please refer to the [WPILib SysId documentation](#) for more details.**

由于 AdvantageKit 已经要求 subsystem 记录相关传感器数据，在用户代码中设置 SysId 测试程序会简化很多。本文介绍使用 AdvantageKit 记录日志时，SysId 数据采集流程有哪些不同。[更多细节请参阅 WPILib SysId 文档。](#)

TIP / 提示

Device logging systems like AdvantageScope's [URCL](#) (Unofficial REV-Compatible Logger) and CTRE's [signal logger](#) can be used to collect data instead of AdvantageKit. In this case, please follow the instructions in the corresponding documentation.

AdvantageScope 的 [URCL](#) (Unofficial REV-Compatible Logger, 非官方 REV 兼容日志记录器) 以及 CTRE 的 [信号日志记录器](#) 等设备日志系统，可以代替 AdvantageKit 采集数据。在这种情况下，请遵循相应文档中的说明。

Code Setup / 代码设置

Create the SysIdRoutine based on the template shown below. Note that the test state is logged as an output through AdvantageKit and the log consumer is set to null. This configuration can be performed within the subsystem class.

根据下方模板创建 SysIdRoutine。请注意，测试状态通过 AdvantageKit 记录为输出，日志 consumer 设置为 null。可以在 subsystem 类内部完成此配置。

```
// Create the SysId routine
var sysIdRoutine = new SysIdRoutine(
    new SysIdRoutine.Config(
        null, null, null, // Use default config
        (state) -> Logger.recordOutput("SysIdTestState", state.toString())
    ),
    new SysIdRoutine.Mechanism(
        (voltage) -> subsystem.runVolts(voltage.in(Volts)),
        null, // No log consumer, since data is recorded by AdvantageKit
        subsystem
    )
);

// The methods below return Command objects
sysIdRoutine.quasistatic(SysIdRoutine.Direction.kForward);
sysIdRoutine.quasistatic(SysIdRoutine.Direction.kReverse);
sysIdRoutine.dynamic(SysIdRoutine.Direction.kForward);
sysIdRoutine.dynamic(SysIdRoutine.Direction.kReverse);
```

Run the SysId routines as normal (described [here](#)), then download the AdvantageKit log file from the robot.

按常规方式运行 SysId 测试程序（步骤见[此处](#)），然后从机器人下载 AdvantageKit 日志文件。

Loading Data / 加载数据

WARNING / 警告

AdvantageKit log files **should NOT be used directly with SysId**. Follow the instructions below to convert it to the correct format.

AdvantageKit 日志文件**不应直接用于 SysId**。请按照以下说明将其转换为正确的格式。

AdvantageKit synchronizes all log updates to the robot loop cycle to enable replay. However, only changes to each field are recorded directly to the log file; by saving the timestamps of each loop cycle, the full set of timestamps where a field was originally recorded can be recreated during replay. This design was chosen because it significantly reduces file size, but it is not compatible with WPILib's SysId analyzer (where explicit updates are expected for every sample, regardless of whether the value changed).

AdvantageKit 会将所有日志更新与机器人循环周期同步，以实现回放。不过，只有每个字段的变化会直接记录到日志文件中；通过保存每个循环周期的时间戳，可以在回放期间重建该字段最初被记录时的完整时间戳集合。选择这种设计是因为它能显著减小文件体积，但它与 WPILib 的 SysId 分析器不兼容（后者要求每个采样点都显式更新，无论值是否发生变化）。

To convert the AdvantageKit log file to a SysId-compatible format, follow the instructions below:

要将 AdvantageKit 日志文件转换为与 SysId 兼容的格式，请按以下步骤操作：

1. Open the AdvantageKit log file in AdvantageScope v3.0.2 or later. In the menu bar, go to "File" > "Export Data...".
在 AdvantageScope v3.0.2 或更高版本中打开 AdvantageKit 日志文件。在菜单栏中选择 "File" > "Export Data..."。
2. Set the format to "WPILOG" and the timestamps to "AdvantageKit Cycles". For large log files, enter the prefixes for only the fields and tables necessary for SysId analysis (see the [export options](#) documentation for details).
将格式设为 "WPILOG"，时间戳设为 "AdvantageKit Cycles"。对于大型日志文件，仅输入 SysId 分析所需字段和表的前缀（详情请参阅[导出选项文档](#)）。
3. Click the save icon and choose a location to save the log.
点击保存图标，选择保存日志的位置。
4. Open the SysId analyzer by searching for "WPILib: Start Tool" in the VSCode command palette and choosing "SysId" (or using the desktop launcher on Windows). Open the exported log file by clicking "Open data log file..."
在 VSCode 命令面板中搜索 "WPILib: Start Tool" 并选择 "SysId"，以打开 SysId 分析器（或在 Windows 上使用桌面启动器）。点击 "Open data log file..." 打开导出的日志文件。
5. Choose the fields to analyze as normal.
按常规方式选择要分析的字段。